

1

Project context & stack

Always include this section at the start of every prompt

Stack

Frontend: React 18 (Vite), React Router v6, Axios, Zustand Backend: Node.js 20, Express.js, ws (WebSocket library) Database: PostgreSQL 15, Knex.js (no ORM) Cache: Redis 7 (ioredis) Auth: JWT (jsonwebtoken) + bcrypt Testing: Jest + Supertest | Vitest + React Testing Library

Project structure

```
/typeracer /frontend/src /pages Auth Game Results Leaderboard Admin /components
TypingBox WpmDisplay Timer ProgressBar /hooks useTyping useWpm useTimer useSocket
/context AuthContext GameContext /services api.js (axios instance with interceptors)
/utils wpmCalc.js keyCapture.js /backend/src /routes auth.js games.js results.js
leaderboard.js /controllers AuthController GameController ResultController /services
AuthService GameService ResultService LeaderboardService /middleware authGuard.js
adminOnly.js rateLimiter.js /db knex.js /migrations /seeds /utils wpm.js jwt.js
socket.js
```

Database tables

```
users id(uuid) email password_hash username role('user'|'admin') created_at games
id(uuid) paragraph(text) difficulty('easy'|'medium'|'hard') created_by created_at
updated_at results id(uuid) user_id(fk) game_id(fk) wpm(float) accuracy(float)
time_taken_ms(int) created_at
```

Always follow this structure. Never use ORMs. Always write raw SQL via Knex.

2

Auth — register, login, JWT, guards

3 backend endpoints + AuthContext + Auth page

Backend

```
POST /auth/register Validate: email format, username 3-20 chars, password min 8 chars
bcrypt hash (saltRounds:12), insert into users Return: { token,
user:{id,email,username,role} } | 409 if email exists POST /auth/login Find user by
email, compare bcrypt hash Sign JWT: payload{userId,role}, expiresIn:'7d',
secret:process.env.JWT_SECRET Return: { token, user:{id,email,username,role} } | 401
on wrong credentials GET /auth/me Protected by authGuard. Return current user from
req.user middleware/authGuard.js Extract Bearer token, verify JWT, attach decoded
payload to req.user Return 401 if missing/invalid/expired middleware/adminOnly.js
Check req.user.role === 'admin', return 403 if not
```

Frontend

```
pages/Auth.jsx Toggle between Login / Signup forms. Controlled inputs + inline
validation. On success: store token in AuthContext + localStorage, redirect to /game
On error: show API error message below the form context/AuthContext.jsx Provide: {
user, token, login(data), logout(), isAdmin } On mount: read token from localStorage,
verify via GET /auth/me logout() clears localStorage and redirects to /auth
```

Return all files with full implementations — no placeholders.

3

Game page — typing engine, WPM, WebSocket

Core feature — 4 hooks + Game page + WebSocket server

Backend

```
GET /games/random?difficulty=medium Select random game row from PostgreSQL. Protected
by authGuard. WebSocket server (backend/src/utils/socket.js) On connect: client sends
{ type:'START', gameId, userId } Server: sends { type:'TICK', elapsed_ms } every 500ms
via setInterval On message: { type:'WPM_UPDATE', wpm, accuracy } -> broadcast to game
room On disconnect: clear interval, remove from room
```

Frontend hooks

```
hooks/useTyping.js Takes: paragraph(string) State: typed, errors(Set of wrong-index
positions), isComplete On each keystroke: compare typed[i] with paragraph[i], track
error positions Returns: { typed, errors, isComplete, handleKeyDown, reset }
hooks/useWpm.js Takes: typed(string), startTime(Date|null) wpm = typed.split('
').length / elapsed_minutes accuracy = correct_chars / total_typed * 100 Returns: {
wpm, accuracy } – recalculates on every typed change hooks/useTimer.js State:
elapsed(ms), running(bool) | start(), pause(), reset() Use useRef for interval to
avoid stale closures hooks/useSocket.js Connect to ws://localhost:4000 on mount,
disconnect on unmount Returns: { sendMessage(obj), lastMessage }
```

Game page

```
pages/Game.jsx On load: fetch GET /games/random Character coloring: Correct -> green
text Wrong -> red background + white text Untyped -> default color Cursor -> blinking
underline Show live WPM + accuracy updating every keystroke Show countdown timer
(starts on first keystroke) On isComplete: POST /results { gameId, wpm, accuracy,
time_taken_ms } Redirect to /results/:id TypingBox: hidden captures input, overlaid
with styled paragraph
```

All hooks must be pure — no direct DOM manipulation. Use refs for focus management only.

4

Results — save, retrieve, display

3 backend endpoints + Results page with replay view

Backend

```
POST /results (authGuard) Body: { gameId, wpm, accuracy, time_taken_ms } Validate:
wpm>0, accuracy 0-100, time_taken_ms>0 Insert into results, return row with game
paragraph joined After insert: DEL Redis key 'leaderboard:top50' GET /results/:id
(authGuard) Return result joined with game.paragraph + user.username Return 403 if
result.user_id !== req.user.userId GET /results/history (authGuard) All results for
req.user.userId, ORDER BY created_at DESC Knex: SELECT results.*, games.paragraph,
games.difficulty FROM results JOIN games ON games.id=results.game_id Paginate:
?page=1&limit;=10
```

Frontend

```
pages/Results.jsx On mount: fetch GET /results/:id using useParams() Display: Large
WPM number (headline stat) Accuracy with color indicator: green>=95% amber>=80%
red<80% Time taken formatted as MM:SS Paragraph replay: correct/wrong chars
highlighted Personal best comparison: 'Your best WPM is X — this is Y% better/worse'
Two buttons: 'Play again' (new random game) | 'View leaderboard'
```

Use Knex joins — never fetch related data in a second query. One query per endpoint.

5

Leaderboard — cached, filtered, ranked

Redis cache-first with PostgreSQL fallback

Backend

```
GET /leaderboard?difficulty=all&limit;=50 (public — no auth) Cache HIT: return Redis
key 'leaderboard:{difficulty}:{limit}' immediately Cache MISS: SELECT DISTINCT ON
(r.user_id) u.username, r.wpm, r.accuracy, g.difficulty FROM results r JOIN users u ON
u.id=r.user_id JOIN games g ON g.id=r.game_id WHERE difficulty filter (if not 'all')
ORDER BY r.user_id, r.wpm DESC -- best result per user Outer sort by wpm DESC, LIMIT N
Store in Redis with SETEX TTL=60s Each row: { rank(1..N), username, wpm, accuracy,
difficulty }
```

Frontend

pages/Leaderboard.jsx Filter bar: difficulty tabs (All / Easy / Medium / Hard) Table columns: Rank Username WPM Accuracy Difficulty Highlight current user's row (compare with AuthContext username) Top 3 rows: gold / silver / bronze rank badge Loading state: 10 gray skeleton rows while fetching Empty state: 'No results yet - be the first!'

Use ioredis client initialized once in backend/src/db/redis.js. Never query PostgreSQL if Redis cache exists.

6

Admin panel — CRUD games

Role-gated feature — admin only

Backend (all routes: authGuard + adminOnly)

GET /games All games paginated (?page=1&limit;=20) ORDER BY created_at DESC POST /games Body:{paragraph,difficulty} Validate: paragraph 80-500 chars, difficulty in [easy,medium,hard] PATCH /games/:id Body:{paragraph?,difficulty?} partial update, return updated row DELETE /games/:id Hard delete, return 204 After any write: SCAN + DEL all 'leaderboard:*' Redis keys

Frontend

pages/Admin.jsx Route guard: if !isAdmin redirect to / with toast 'Access denied' Game list table: paragraph(60-char truncate) difficulty badge created_at actions Actions per row: Edit (pencil icon) Delete (trash icon) 'Add game' button: opens inline form above table (not a modal) Inline form fields: paragraph with live char counter (red if out of 80-500 range) difficulty: easy / medium / hard Submit Cancel Edit mode: pre-fills form, switches Submit label to 'Update' Delete: inline confirmation 'Are you sure? This cannot be undone' -> Confirm/Cancel On success: refetch list, show success toast (build useToast hook)

No external UI component libraries. Build all UI with plain HTML + CSS-in-JS style props.

7

Infrastructure — env, migrations, error handling

Ship-ready setup — run this section first

Environment files

```
backend/.env.example PORT=4000
DATABASE_URL=postgresql://user:pass@localhost:5432/typeracer
REDIS_URL=redis://localhost:6379 JWT_SECRET=your_jwt_secret_here BCrypt_SALT_ROUNDS=12
NODE_ENV=development frontend/.env.example VITE_API_BASE_URL=http://localhost:4000
VITE_WS_URL=ws://localhost:4000
```

Knex migrations (backend/src/db/migrations/)

```
001_create_users.js users table + indexes 002_create_games.js games table
003_create_results.js results table + foreign keys + indexes: CREATE INDEX ON
results(user_id) CREATE INDEX ON results(wpm DESC) CREATE INDEX ON results(game_id)
```

Knex seeds (backend/src/db/seeds/)

```
01_admin_user.js email:admin@typeracer.com password:Admin1234! role:admin
02_sample_games.js 10 paragraphs across easy / medium / hard
```

Error handling

```
backend/src/middleware/errorHandler.js Express global error handler (4-arg middleware)
ValidationError->400 UnauthorizedError->401 ForbiddenError->403 NotFoundError->404
Development: include stack trace | Production: generic message only Log all 5xx errors
to console.error with timestamp frontend/src/services/api.js Axios instance with
baseUrl from import.meta.env.VITE_API_BASE_URL Request interceptor: attach
Authorization: Bearer {token} from localStorage Response interceptor: on 401, clear
localStorage and redirect to /auth knexfile.js Configure development + production
connections using DATABASE_URL
```

Provide complete runnable code for every file listed above.